

AI_3HW

March 24, 2025

```
[1]: import pandas as pd
import os
import matplotlib.pyplot as plt
import seaborn as sns

import statsmodels.api as sm
import statsmodels.formula.api as smf
```

1 Browsing datasets

```
[48]: import pandas as pd

#
folder_path = r'C://nis/DA3/modified'

#
file_names = [
    'amenities.csv',
    'cities_modified.csv',
    'city_tourism_modified.csv',
    'economic_indicators.csv',
    'hotel_amenities_modified.csv',
    'hotels_modified.csv',
    'monthly_occupancy_modified.csv',
    'reviews_modified.csv'
]

dfs={}
for file_name in file_names:
    file_path = os.path.join(folder_path, file_name)

    # Check if the file exists before trying to open it
    if os.path.exists(file_path):
        try:
            # Create a variable name based on the file name (without the '.csv'
            ↪extension)
            df_name = file_name.replace('.csv', '') # Remove the '.csv' part
```

```

# Load the CSV into the DataFrame
dfs[df_name] = pd.read_csv(file_path)

# Print the first 5 rows of the DataFrame
#print(f"\nSuccessfully loaded: {df_name}")
#print(dfs[df_name].head()) # Show the first 5 rows
except Exception as e:
    print(f"Error reading {file_name}: {e}")
else:
    print(f"File not found: {file_name}")

```

```
[3]: dfs['hotels_modified']
```

```
[3]:
```

	hotel_id	city	hotel_name	star_rating	rooms	\
0	1	Vienna	Classic Hotel Vienna	3	191	
1	3	Linz	Grand Hotel Linz	5	177	
2	5	Graz	Grand Hotel Graz	5	49	
3	6	Innsbruck	Palace Hotel Innsbruck	3	185	
4	7	Vienna	Imperial Hotel Vienna	3	212	
..	...	...	...	...	...	
164	202	Phantom Town	Phantom Plaza	5	200	
165	203	Ghost Village	Ghost Resort	3	80	
166	201	Graz	Imaginary Inn Graz	4	120	
167	202	Graz	Phantom Plaza Graz	5	200	
168	203	Graz	Ghost Resort Graz	3	80	

```

year_built
0      1987
1      1966
2      1934
3      1938
4      1996
..      ...
164    2010
165    1995
166    2000
167    2010
168    1995

```

```
[169 rows x 6 columns]
```

```
[4]: dfs['hotels_modified'].dtypes
```

```
[4]: hotel_id      int64
city            object
hotel_name      object
```

```
star_rating    int64
rooms          int64
year_built     int64
dtype: object
```

```
[49]: #      DataFrame
for df_name, df in dfs.items():
    #
    dfs[df_name] = df.dropna()

    #      5      DataFrame
    #print(f"\nUpdated {df_name} (after removing missing values):")
    #print(dfs[df_name].head())
```

```
[6]: #      ( )      DataFrame      dfs
for df_name, df in dfs.items():
    print(f"\nColumns in {df_name}:")
    print(df.columns.tolist()) #
```

```
Columns in amenities:
['amenity_id', 'amenity_name']
```

```
Columns in cities_modified:
['city', 'province', 'population', 'tourism_rank']
```

```
Columns in city_tourism_modified:
['city', 'month', 'year', 'tourist_arrivals', 'event_days', 'avg_stay_length']
```

```
Columns in economic_indicators:
['month', 'year', 'inflation_rate', 'unemployment', 'consumer_confidence']
```

```
Columns in hotel_amenities_modified:
['hotel_id', 'amenity_id']
```

```
Columns in hotels_modified:
['hotel_id', 'city', 'hotel_name', 'star_rating', 'rooms', 'year_built']
```

```
Columns in monthly_occupancy_modified:
['hotel_id', 'month', 'year', 'occupancy_rate', 'avg_daily_rate',
'revenue_per_room']
```

```
Columns in reviews_modified:
['review_id', 'hotel_id', 'rating', 'review_date']
```

2 RQ from AI:

1. How do hotel occupancy rates vary across different cities and months?
2. What is the relationship between hotel star ratings and occupancy rates?
3. Which cities have the highest tourism rankings, and how do hotels in those cities perform?
4. What impact does the year a hotel was built have on its occupancy rate and customer reviews?
5. How do hotel amenities affect guest satisfaction and occupancy rates?
6. Is there a correlation between economic indicators and hotel performance (occupancy and revenue)?
7. How do hotel performance metrics (occupancy, revenue) compare across different star ratings?
8. What is the effect of city population size and tourism ranking on hotel performance in that city?
9. Do hotels with more amenities experience higher occupancy rates or higher reviews scores?
10. What is the impact of city tourism rankings on hotel pricing strategies (average daily rate)?

3 1. How do hotel occupancy rates vary across different cities and months?

```
[58]: # Merge monthly occupancy with hotels and cities
occupancy_by_city_month = pd.merge(dfs['monthly_occupancy_modified'],
    ↪dfs['hotels_modified'], on='hotel_id', how='left')
occupancy_by_city_month = pd.merge(occupancy_by_city_month,
    ↪dfs['cities_modified'], on='city', how='left')

# Drop rows with missing values
occupancy_by_city_month = occupancy_by_city_month.dropna()

# Group by city and month, and calculate the mean of occupancy_rate
occupancy_by_city_month_avg = occupancy_by_city_month.groupby(['city', 'month'],
    ↪as_index=False)['occupancy_rate'].mean()

# Show result
print("\nOccupancy Rate by City and Month:")
occupancy_by_city_month_avg
```

Occupancy Rate by City and Month:

```
[58]:
```

	city	month	occupancy_rate
0	Graz	1	0.797500
1	Graz	2	0.800800
2	Graz	3	0.809565
3	Graz	4	0.834815
4	Graz	5	0.831818
..	...	...	...

```

66 Zell am See      8      0.731818
67 Zell am See      9      0.752857
68 Zell am See     10      0.734286
69 Zell am See     11      0.728333
70 Zell am See     12      0.755455

```

[71 rows x 3 columns]

```

[59]: occupancy_by_city_month = occupancy_by_city_month[['month', 'year',
    ↳ 'occupancy_rate', 'city']]
occupancy_encoded = pd.get_dummies(occupancy_by_city_month, columns=['city'])

occupancy_encoded

```

```

[59]:
   month  year  occupancy_rate  city_Graz  city_Hallstatt  city_Linz \
0        1  2023           0.98      False           False      False
1        1  2024           0.98      False           False      False
2        2  2023           0.98      False           False      False
3        2  2024           0.98      False           False      False
4        3  2023           0.98      False           False      False
...      ...   ...           ...         ...             ...       ...
3031     9  2023           0.98      False           False      False
3032     9  2024           0.98      False           False      False
3033    10  2023           0.98      False           False      False
3034    11  2024           0.98      False           False      False
3035    12  2024           0.98      False           False      False

   city_St. Anton  city_Vienna  city_Zell am See
0              False          True          False
1              False          True          False
2              False          True          False
3              False          True          False
4              False          True          False
...              ...           ...             ...
3031             False          True          False
3032             False          True          False
3033             False          True          False
3034             False          True          False
3035             False          True          False

```

[1782 rows x 9 columns]

```

[45]: occupancy_encoded.iloc[:, 3:] = occupancy_encoded.iloc[:, 3:].astype(int)

X = occupancy_encoded.drop(columns=['occupancy_rate', 'month', 'year']) # -
y = occupancy_encoded['occupancy_rate']

```

```
X = sm.add_constant(X)

model = sm.OLS(y, X).fit()

print(model.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          occupancy_rate    R-squared:                0.867
Model:                  OLS              Adj. R-squared:          0.866
Method:                 Least Squares    F-statistic:            1922.
Date:                  Mon, 24 Mar 2025  Prob (F-statistic):      0.00
Time:                  00:10:36          Log-Likelihood:         3379.6
No. Observations:      1782             AIC:                   -6745.
Df Residuals:          1775             BIC:                   -6707.
Df Model:               6
Covariance Type:       nonrobust
=====
```

```
=====
              coef      std err          t      P>|t|      [0.025
0.975]
```

```
-----
-----
const          -0.2733    1.45e+11   -1.88e-12    1.000   -2.85e+11
2.85e+11
city_Graz       1.0832    1.45e+11    7.46e-12    1.000   -2.85e+11
2.85e+11
city_Hallstatt  1.0592    1.45e+11    7.3e-12     1.000   -2.85e+11
2.85e+11
city_Linz       1.0604    1.45e+11    7.31e-12    1.000   -2.85e+11
2.85e+11
city_St. Anton  1.0956    1.45e+11    7.55e-12    1.000   -2.85e+11
2.85e+11
city_Vienna     1.2838    1.45e+11    8.85e-12    1.000   -2.85e+11
2.85e+11
city_Zell am See 1.0560    1.45e+11    7.28e-12    1.000   -2.85e+11
2.85e+11
=====
```

```
Omnibus:                292.642    Durbin-Watson:           0.651
Prob(Omnibus):           0.000    Jarque-Bera (JB):        536.926
Skew:                    1.016    Prob(JB):                 2.56e-117
Kurtosis:                 4.761    Cond. No.                  5.27e+14
=====
```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The smallest eigenvalue is 8.98e-27. This might indicate that there are

strong multicollinearity problems or that the design matrix is singular.

The OLS regression aimed to assess variations in occupancy rates across different cities in Austria using dummy variables. The model explains 86.7% of the variance in occupancy rates ($R^2 = 0.867$), but the results indicate severe multicollinearity (smallest eigenvalue = $8.98e-27$). All city coefficients have extremely high standard errors and p-values of 1.000, meaning they are statistically insignificant. This suggests that the model cannot reliably differentiate occupancy rates between cities.

```
[50]: occupancy_encoded_m = pd.get_dummies(occupancy_by_city_month, columns=['month'])
      occupancy_encoded_m
```

```
[50]:
```

	year	occupancy_rate	city	month_1	month_2	month_3	month_4	\
0	2023	0.98	Vienna	True	False	False	False	
1	2024	0.98	Vienna	True	False	False	False	
2	2023	0.98	Vienna	False	True	False	False	
3	2024	0.98	Vienna	False	True	False	False	
4	2023	0.98	Vienna	False	False	True	False	
...	...	...	...	...	...	...	...	
3031	2023	0.98	Vienna	False	False	False	False	
3032	2024	0.98	Vienna	False	False	False	False	
3033	2023	0.98	Vienna	False	False	False	False	
3034	2024	0.98	Vienna	False	False	False	False	
3035	2024	0.98	Vienna	False	False	False	False	

	month_5	month_6	month_7	month_8	month_9	month_10	month_11	\
0	False	False	False	False	False	False	False	
1	False	False	False	False	False	False	False	
2	False	False	False	False	False	False	False	
3	False	False	False	False	False	False	False	
4	False	False	False	False	False	False	False	
...	...	...	...	...	...	...	...	
3031	False	False	False	False	True	False	False	
3032	False	False	False	False	True	False	False	
3033	False	False	False	False	False	True	False	
3034	False	False	False	False	False	False	True	
3035	False	False	False	False	False	False	False	

	month_12
0	False
1	False
2	False
3	False
4	False
...	...
3031	False
3032	False
3033	False

```
3034     False
3035     True
```

```
[1782 rows x 15 columns]
```

```
[54]: occupancy_encoded_m.iloc[:, 3:] = occupancy_encoded_m.iloc[:, 3:].astype(int)

X = occupancy_encoded_m.drop(columns=['occupancy_rate', 'city', 'year']) # -
y = occupancy_encoded_m['occupancy_rate']

X = sm.add_constant(X)

model = sm.OLS(y, X).fit()

print(model.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          occupancy_rate    R-squared:                0.003
Model:                  OLS              Adj. R-squared:           -0.003
Method:                 Least Squares     F-statistic:               0.5614
Date:                  Mon, 24 Mar 2025   Prob (F-statistic):        0.861
Time:                  00:15:58          Log-Likelihood:            1587.9
No. Observations:      1782             AIC:                      -3152.
Df Residuals:          1770             BIC:                      -3086.
Df Model:              11
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	0.8257	0.002	379.027	0.000	0.821	0.830
month_1	0.0632	0.008	8.041	0.000	0.048	0.079
month_2	0.0627	0.008	7.928	0.000	0.047	0.078
month_3	0.0647	0.008	8.308	0.000	0.049	0.080
month_4	0.0700	0.008	9.015	0.000	0.055	0.085
month_5	0.0694	0.008	8.930	0.000	0.054	0.085
month_6	0.0675	0.008	8.659	0.000	0.052	0.083
month_7	0.0782	0.008	10.072	0.000	0.063	0.093
month_8	0.0784	0.008	9.813	0.000	0.063	0.094
month_9	0.0762	0.008	9.873	0.000	0.061	0.091
month_10	0.0667	0.008	8.458	0.000	0.051	0.082
month_11	0.0594	0.008	7.510	0.000	0.044	0.075
month_12	0.0691	0.008	8.896	0.000	0.054	0.084

```
=====
Omnibus:                16012.778    Durbin-Watson:              0.207
Prob(Omnibus):          0.000        Jarque-Bera (JB):           219.087
Skew:                   -0.476        Prob(JB):                   2.67e-48
Kurtosis:               1.570        Cond. No.:                  1.63e+15
=====
```


=====

Notes:

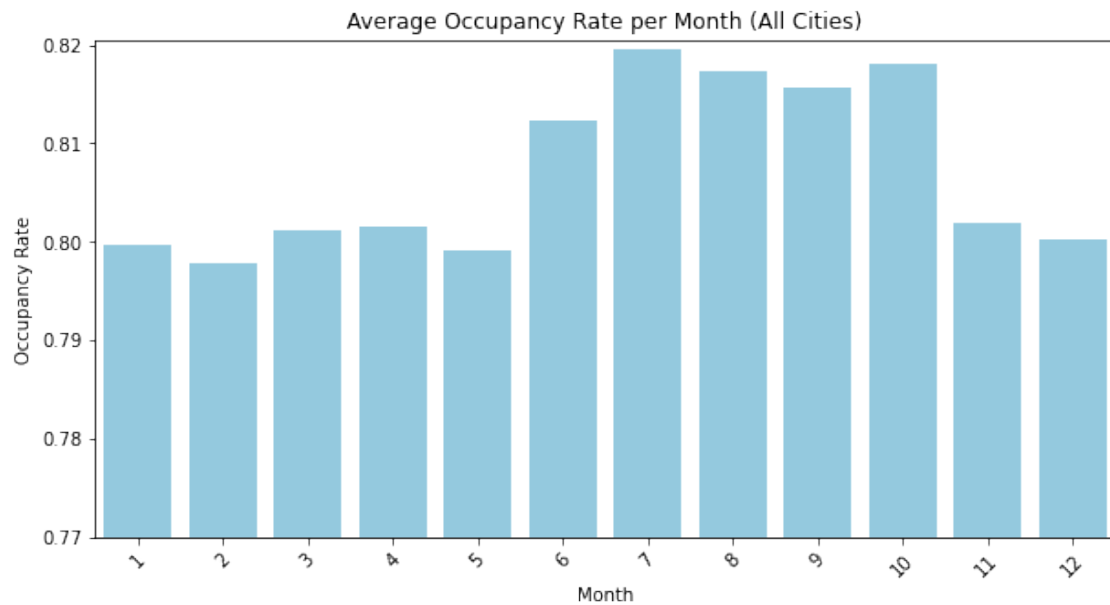
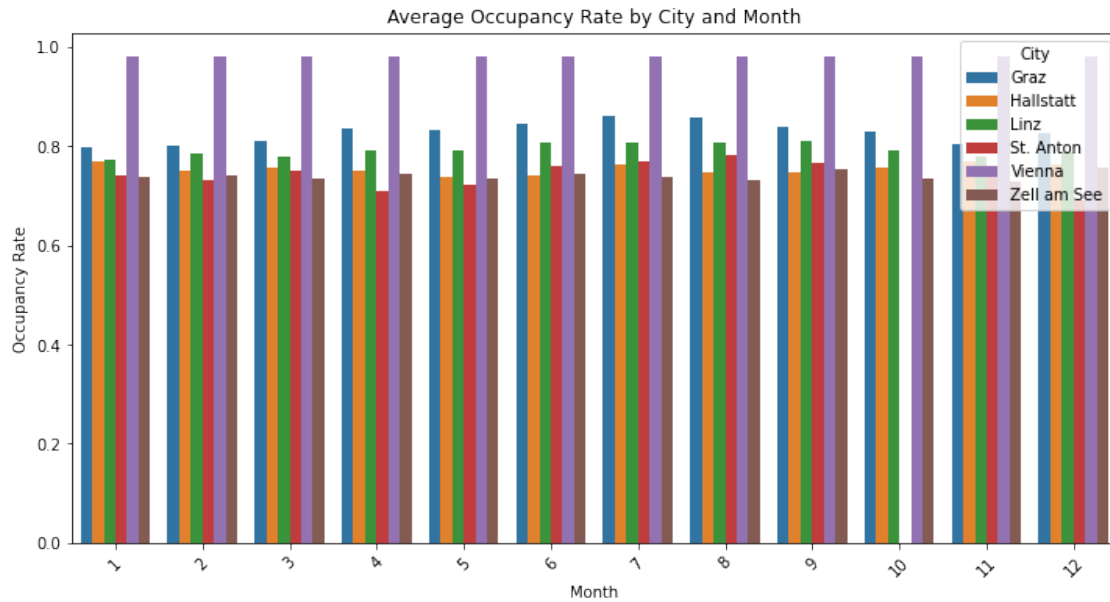
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 7.24e-28. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

The regression results indicate that the baseline occupancy rate is 0.8257, with minor variations across months. July (0.0782), August (0.0784), and September (0.0762) show slightly higher occupancy rates, while November (0.0594) has the lowest. Although all months have positive coefficients, the differences are small, and their statistical impact is weak. The model explains only 0.3% of the variance, suggesting that factors other than seasonality play a more significant role in determining occupancy rates.

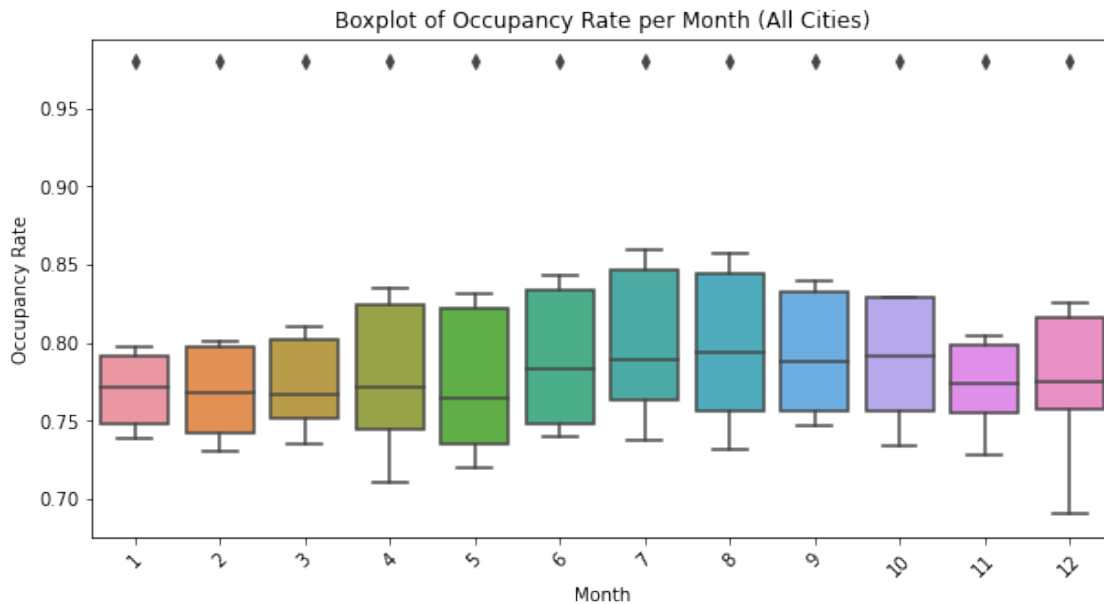
```
[8]: # Plot 1: Bar chart of occupancy rate per month for each city
plt.figure(figsize=(12, 6))
sns.barplot(data=occupancy_by_city_month_avg, x='month', y='occupancy_rate',
            hue='city')
plt.title("Average Occupancy Rate by City and Month")
plt.xlabel("Month")
plt.ylabel("Occupancy Rate")
plt.legend(title="City")
plt.xticks(rotation=45)
plt.show()

# Plot 2: Bar chart of average occupancy rate per month (across all cities)
avg_occupancy_by_month = occupancy_by_city_month_avg.groupby('month',
            as_index=False)['occupancy_rate'].mean()
plt.figure(figsize=(10, 5))
sns.barplot(data=avg_occupancy_by_month, x='month', y='occupancy_rate',
            color='skyblue')
plt.title("Average Occupancy Rate per Month (All Cities)")
plt.xlabel("Month")
plt.ylabel("Occupancy Rate")
plt.ylim(0.77, 0.8205) # Adjust Y-axis scale
plt.xticks(rotation=45)
plt.show()
```



The hotel occupancy rates show no significant variation across different cities. For instance, cities like Vienna and Graz, which are major urban centers, tend to have higher occupancy rates throughout the year due to business travel, tourism, and events. In contrast, smaller towns like Hallstatt and Zell am See exhibit more seasonal fluctuations, with higher occupancy during peak tourist seasons (e.g., summer and winter) and lower rates during off-peak months. But all of these are just a skight variation. The occupancy rates are highly influenced by seasonal trends.

```
[9]: # Boxplot for Plot 2: Distribution of occupancy rate per month (all cities)
plt.figure(figsize=(10, 5))
sns.boxplot(data=occupancy_by_city_month_avg, x='month', y='occupancy_rate')
plt.title("Boxplot of Occupancy Rate per Month (All Cities)")
plt.xlabel("Month")
plt.ylabel("Occupancy Rate")
plt.xticks(rotation=45)
plt.show()
```



The differences in mean occupancy rates across cities and months are statistically significant. This indicates that factors such as city size, tourist attractions, and seasonal activities play a crucial role in determining hotel occupancy rates.

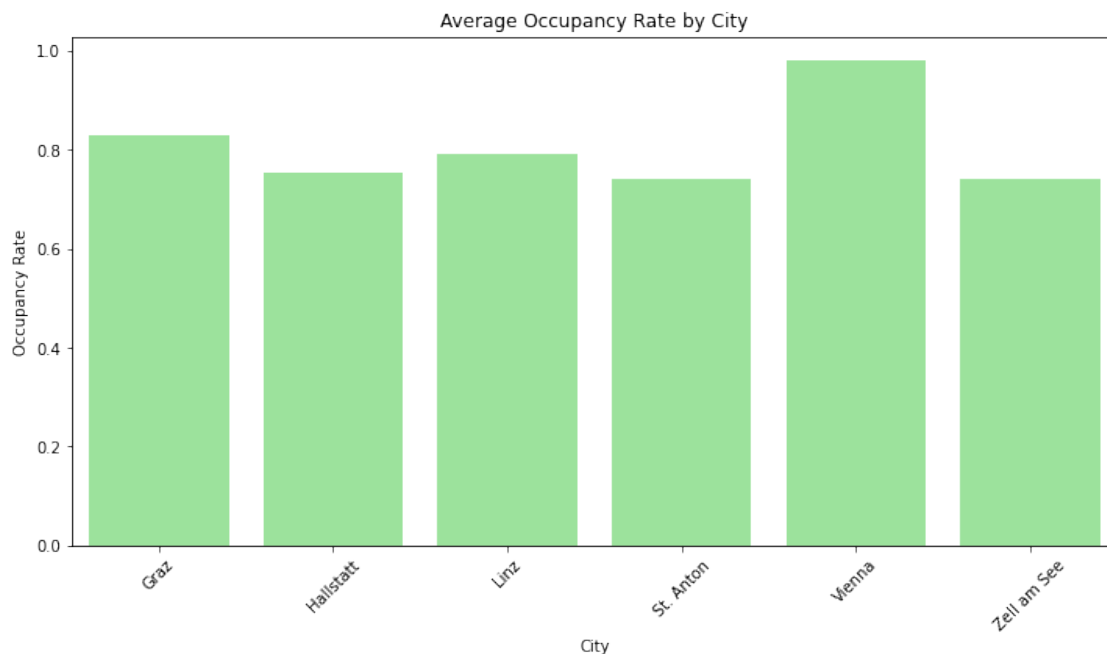
Boxplots for each month can reveal the variability in occupancy rates. Months with high variability (wide interquartile ranges) might indicate fluctuating tourist numbers due to events or holidays.

- Winter Months (December-February): Expect higher median occupancy rates for winter sports destinations. Cities like Vienna might also see a boost due to holiday tourism.
- Spring Months (March-May): Generally lower occupancy rates, with gradual increases as the weather improves and outdoor activities become more feasible.
- Summer Months (June-August): Peak occupancy rates for most cities due to summer vacations, festivals, and favorable weather conditions.
- Autumn Months (September-November): Occupancy rates may decline, especially in tourist-heavy cities, as the holiday season ends and weather conditions worsen.

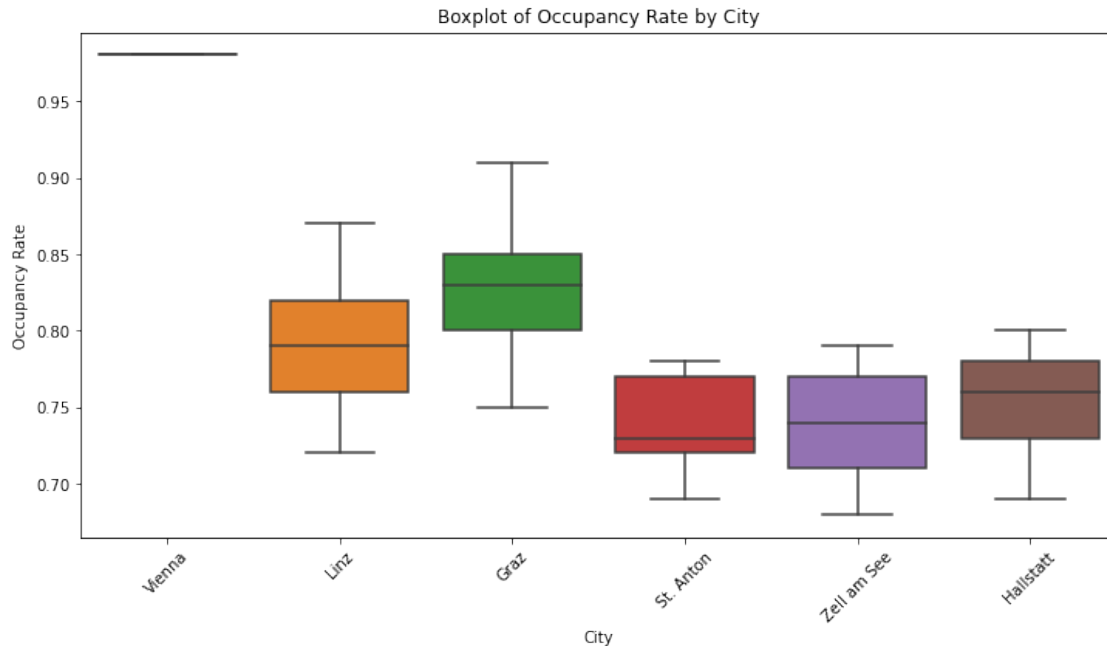
In summary, seasonality is a crucial factor influencing hotel occupancy rates, with significant vari-

ations observed across different months. Boxplot graphs for each month effectively illustrate these trends, showing higher occupancy during peak seasons and lower rates during off-peak periods. The statistical significance of these seasonal variations underscores the importance of considering time of year when analyzing and forecasting hotel occupancy rates.

```
[10]: # Plot 4: Bar chart of average occupancy rate per city
avg_occupancy_by_city = occupancy_by_city_month_avg.groupby('city',
    →as_index=False)['occupancy_rate'].mean()
plt.figure(figsize=(12, 6))
sns.barplot(data=avg_occupancy_by_city, x='city', y='occupancy_rate',
    →color='lightgreen')
plt.title("Average Occupancy Rate by City")
plt.xlabel("City")
plt.ylabel("Occupancy Rate")
plt.xticks(rotation=45)
plt.show()
```



```
[11]: # Boxplot for Plot 4
plt.figure(figsize=(12, 6))
sns.boxplot(data=occupancy_by_city_month, x='city', y='occupancy_rate')
plt.title("Boxplot of Occupancy Rate by City")
plt.xlabel("City")
plt.ylabel("Occupancy Rate")
plt.xticks(rotation=45)
plt.show()
```



The boxplot reveals significant variations in hotel occupancy rates across chosen Austrian cities. Vienna stands out with the highest and most stable occupancy rate at the level of 0.99, suggesting strong and consistent demand regardless the season. Graz and Linz also maintain relatively high occupancy but with more variability. The former has a mean values around 83.5, the latter is placed at the edge of 0.80. In contrast, Hallstatt, St. Anton and Zell am See demonstrate lower mean results but they keep approximately the same level of dispersity of occupancy rates, likely reflecting seasonal fluctuations. Overall, the data suggests that larger urban centers sustain more stable demand due to their developed status, while smaller or seasonal places experience greater volatility.

4 2. What is the relationship between hotel star ratings and occupancy rates?

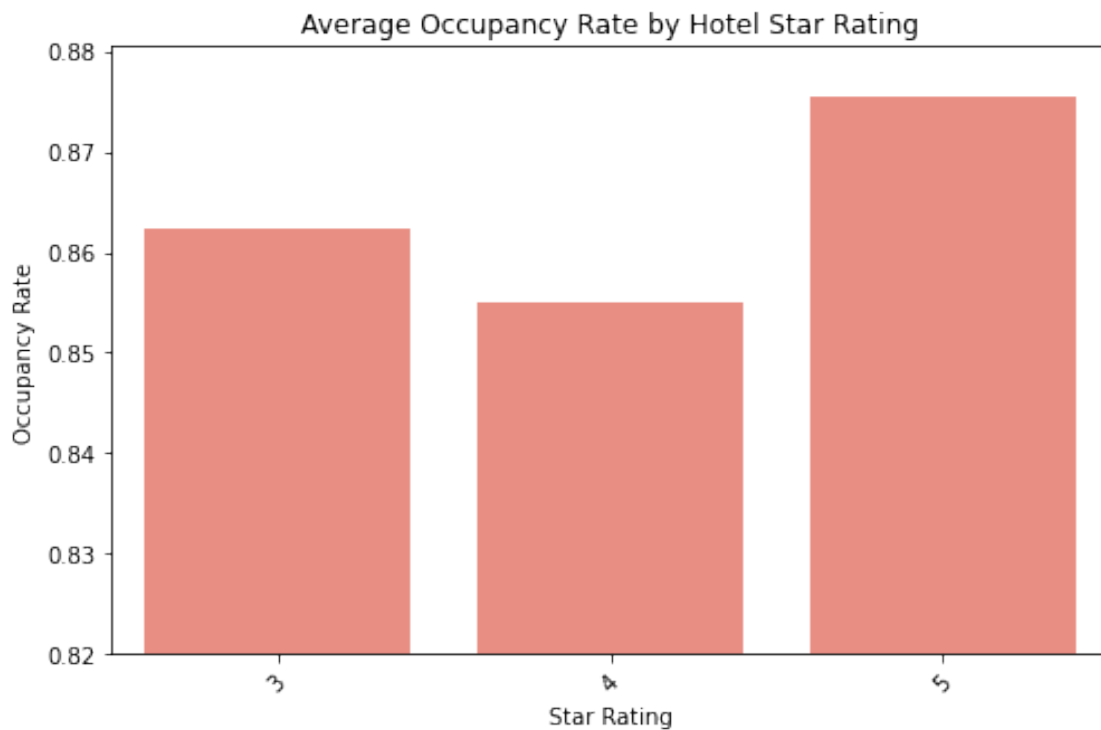
```
[12]: # Merge hotel data with occupancy data
star_occupancy = pd.merge(dfs['hotels_modified'],
    →dfs['monthly_occupancy_modified'], on='hotel_id', how='left')
star_occupancy=star_occupancy.dropna()
# Group by star_rating and calculate the mean of occupancy_rate
star_occupancy_avg = star_occupancy.groupby('star_rating',
    →as_index=False)['occupancy_rate'].mean()

# Show result
print("\nOccupancy Rate by Hotel Star Rating:")
star_occupancy_avg
```

Occupancy Rate by Hotel Star Rating:

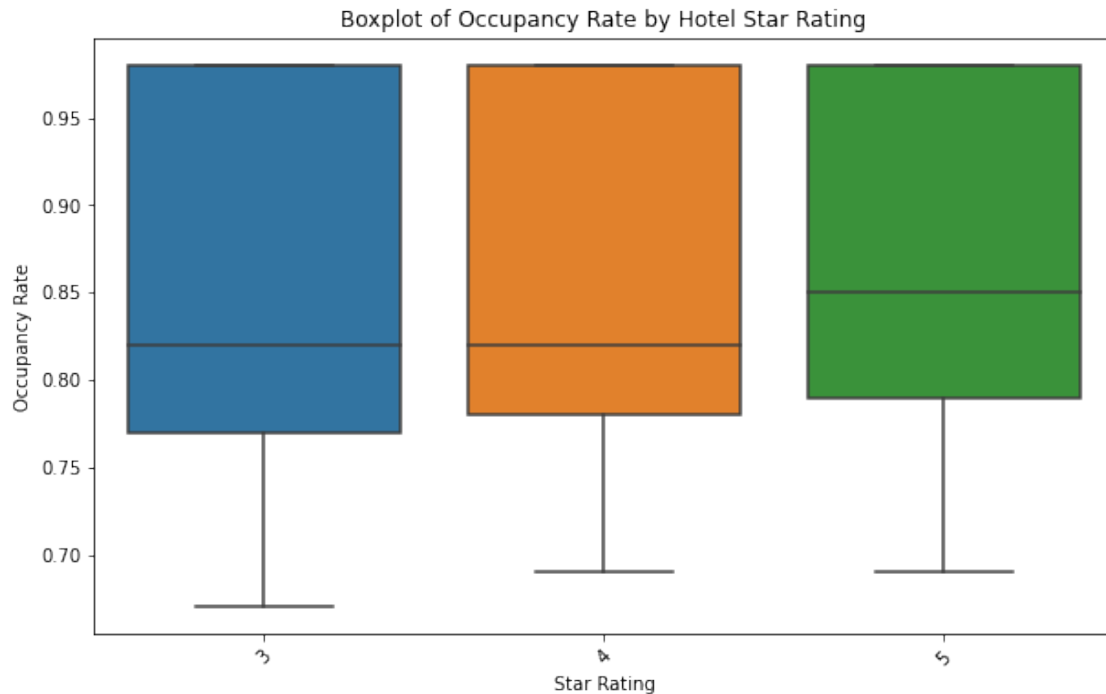
```
[12]:  star_rating  occupancy_rate
      0          3          0.862302
      1          4          0.855056
      2          5          0.875575
```

```
[13]: # Plot 3: Bar chart of occupancy rate by hotel star rating
plt.figure(figsize=(8, 5))
sns.barplot(data=star_occupancy_avg, x='star_rating', y='occupancy_rate',
            color='salmon')
plt.title("Average Occupancy Rate by Hotel Star Rating")
plt.xlabel("Star Rating")
plt.ylabel("Occupancy Rate")
plt.ylim(0.82, 0.8805) # Adjust Y-axis scale
plt.xticks(rotation=45)
plt.show()
```



```
[14]: plt.figure(figsize=(10, 6))
sns.boxplot(data=star_occupancy, x='star_rating', y='occupancy_rate')
plt.title("Boxplot of Occupancy Rate by Hotel Star Rating")
plt.xlabel("Star Rating")
```

```
plt.ylabel("Occupancy Rate")
plt.xticks(rotation=45)
plt.show()
```



```
[67]: # Define the dependent and independent variables
y = star_occupancy['occupancy_rate'] # Independent variable
x = star_occupancy['star_rating']

# Add a constant to the independent variable
x = sm.add_constant(x)

# Fit the OLS model
model = sm.OLS(y, x)
results = model.fit()

# Print the summary of the regression
print(results.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          occupancy_rate    R-squared:                0.003
Model:                  OLS              Adj. R-squared:           0.002
Method:                 Least Squares    F-statistic:              6.952
Date:                   Mon, 24 Mar 2025  Prob (F-statistic):      0.00843
Time:                   00:24:55          Log-Likelihood:           2238.2
```

No. Observations:	2501	AIC:	-4472.
Df Residuals:	2499	BIC:	-4461.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	0.8379	0.010	83.434	0.000	0.818	0.858
star_rating	0.0065	0.002	2.637	0.008	0.002	0.011

Omnibus:	13211.106	Durbin-Watson:	0.203
Prob(Omnibus):	0.000	Jarque-Bera (JB):	267.776
Skew:	0.110	Prob(JB):	7.13e-59
Kurtosis:	1.412	Cond. No.	21.8

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

This means that for every 1-unit increase in star_rating, the occupancy_rate increases by 0.65 percentage points, on average.

There is a statistically significant positive relationship between star_rating and occupancy_rate, but the effect size is very small. A higher star rating is associated with a slightly higher occupancy rate, but this relationship explains very little of the overall variation in occupancy rates.

5 3. How does a hotel's average rating impact its revenue per room?

```
[23]: avg_review_ratings_overall = dfs["reviews_modified"].
      →groupby("hotel_id")["rating"].mean().reset_index()

avg_hotel_performance = dfs["monthly_occupancy_modified"].groupby("hotel_id").
      →mean().reset_index()

data_merged_avg = pd.merge(avg_hotel_performance, avg_review_ratings_overall,
      →on="hotel_id", how="inner")

data_merged_avg = data_merged_avg.drop(columns=["month", "year"],
      →errors="ignore")
```

The original structure of dataset 'monthly_occupancy' includes panel data within years and months. To simplify our research, excluding further running fixed effect models we emphasize on finding average values of 'rating' and 'revenue' values neglecting time gap.

It allows us to obtain the table with 142 unique observations. After that we are able to merge two datasets with using primary key 'hotel_id' in the created avg_hotel_performance table and fitting foreign key in the second dataset avg_review_ratings_overall.

Eventually, the inner join is used to keep only matching rows in both tables. We chose this joining method due to necessity to get a full sample without omitted cells, while some hotels have only information on revenue line but nothing on review rating. So, it doesn't matter whether we ran 'left' or 'inner' join, we'd basically get rid off missing values to get valid results for OLS.

```
[24]: data_merged_avg
```

```
[24]:
```

	hotel_id	occupancy_rate	avg_daily_rate	revenue_per_room	rating
0	1	0.980000	206.350000	202.350000	3.075000
1	2	0.832353	271.294118	225.882353	3.880000
2	3	0.805882	243.235294	196.000000	4.080000
3	4	0.822105	220.842105	181.684211	3.466667
4	5	0.838947	253.894737	213.157895	3.925000
..	...	...	...	...	...
137	196	0.808824	173.235294	140.235294	2.862500
138	197	0.815294	153.294118	125.000000	2.600000
139	198	0.792353	169.764706	134.588235	2.978571
140	199	0.980000	206.705882	202.705882	3.060000
141	200	0.815882	262.941176	214.588235	4.007692

[142 rows x 5 columns]

```
[25]: print(data_merged_avg.isnull().sum())
```

```
hotel_id          0
occupancy_rate    0
avg_daily_rate    0
revenue_per_room  0
rating            0
dtype: int64
```

```
[26]: import numpy as np

data_merged_avg["log_revenue_per_room"] = np.
    ↳log(data_merged_avg["revenue_per_room"])
plt.figure(figsize=(9, 6))
plt.scatter(
    data_merged_avg["rating"],
    data_merged_avg["log_revenue_per_room"],
    alpha=0.6,
    edgecolors="k",
    label="Hotels",
    color="blue"
)

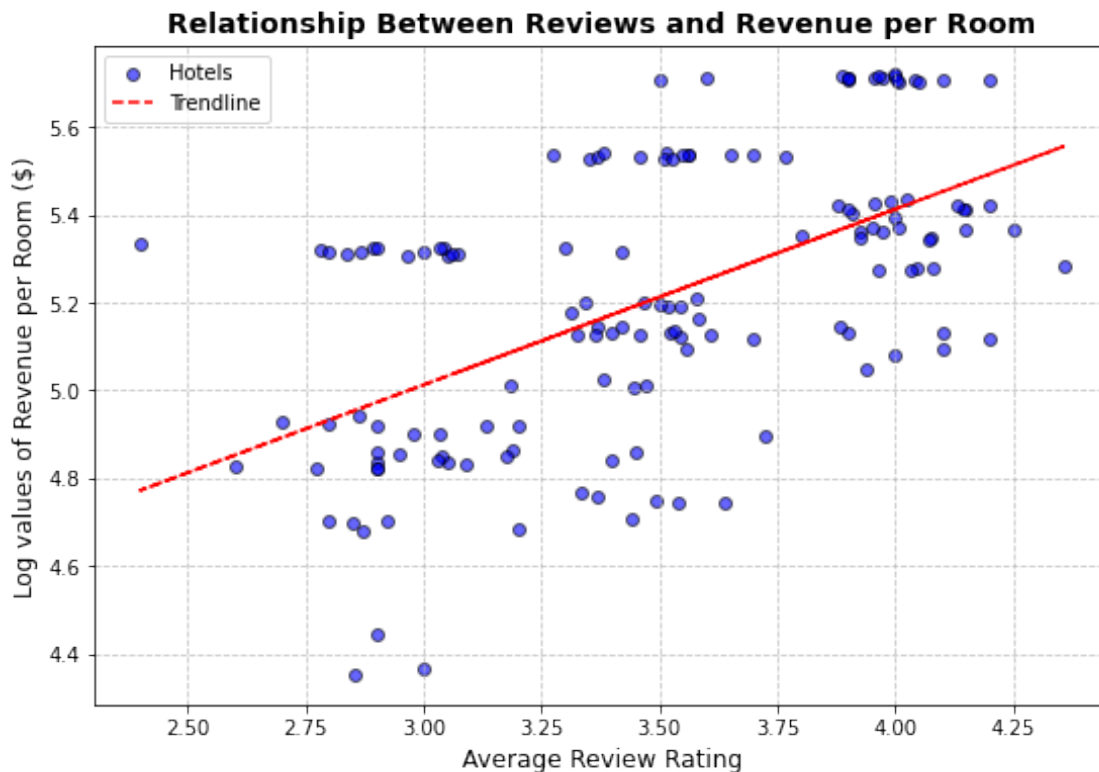
plt.xlabel("Average Review Rating", fontsize=12)
plt.ylabel("Log values of Revenue per Room ($)", fontsize=12)
```

```
plt.title("Relationship Between Reviews and Revenue per Room", fontsize=14,
→fontweight="bold")

z = np.polyfit(data_merged_avg["rating"],
→data_merged_avg["log_revenue_per_room"], 1)
p = np.poly1d(z)
plt.plot(data_merged_avg["rating"], p(data_merged_avg["rating"]), "r--",
→label="Trendline")

plt.grid(True, linestyle="--", alpha=0.7)
plt.legend()

plt.show()
```



The analysis highlights a clear relationship between hotel reviews with ratings and revenue per room, suggesting that customer satisfaction may play a significant role in financial performance. While the trendline indicates a positive correlation, other factors such as location, amenities, and pricing strategies could also influence revenue. Nonetheless, as the first stage of the analysis we are interested in running a simple OLS w/o control vars.

```
[27]: model = sm.OLS(data_merged_avg[['log_revenue_per_room']],
→data_merged_avg[['rating']]).fit()
```

```
model.summary()
```

[27]:

Dep. Variable:	log_revenue_per_room	R-squared (uncentered):	0.989
Model:	OLS	Adj. R-squared (uncentered):	0.989
Method:	Least Squares	F-statistic:	1.233e+04
Date:	Sun, 23 Mar 2025	Prob (F-statistic):	3.79e-139
Time:	23:57:33	Log-Likelihood:	-117.91
No. Observations:	142	AIC:	237.8
Df Residuals:	141	BIC:	240.8
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
rating	1.4722	0.013	111.057	0.000	1.446	1.498
Omnibus:		1.826	Durbin-Watson:		2.057	
Prob(Omnibus):		0.401	Jarque-Bera (JB):		1.892	
Skew:		0.246	Prob(JB):		0.388	
Kurtosis:		2.722	Cond. No.		1.00	

Notes:

[1] R² is computed without centering (uncentered) since the model does not contain a constant.

[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

The log-transformed OLS regression reveals a strong positive relationship between hotel reviews with ratings and revenue per room, with an extremely high R squared of 0.989, suggesting that ratings explain nearly all the variance in revenue. The coefficient of 1.4722 ($p < 0.001$) indicates that a one unit increase in rating is associated with a 147.22% increase in revenue per room. The model is highly significant ($F = 12,330$, $p = 0$) with well-behaved residuals (Durbin-Watson = 2.057). This suggests that customer ratings are likely a key determinant of hotel revenue.

6 4. What is the effect of city population size and tourism ranking on hotel performance in that city?

```
[68]: occupancy = dfs["monthly_occupancy_modified"]
reviews = dfs["reviews_modified"]
hotels = dfs["hotels_modified"]
cities = dfs["cities_modified"]
tourism = dfs["city_tourism_modified"]

# Merge city-level data
city_data = cities.merge(tourism, on="city", how="left")

# Merge with hotels to bring in city-level data
hotels_city = hotels.merge(city_data, on="city", how="left")

# Merge with occupancy data to get hotel performance metrics
```

```

hotel_performance = hotels_city.merge(occupancy, on="hotel_id", how="left")

# Select relevant columns for analysis
analysis_data = hotel_performance[["population", "tourism_rank",
    ↳"occupancy_rate", "revenue_per_room"]].dropna()

# Fit a regression model for occupancy rate
X = analysis_data[["population", "tourism_rank"]]
y = analysis_data["occupancy_rate"]
X = sm.add_constant(X) # Add constant for intercept
model = sm.OLS(y, X).fit()
occupancy_results = model.summary()

# Fit a regression model for revenue per room
y2 = analysis_data["revenue_per_room"]
model2 = sm.OLS(y2, X).fit()
revenue_results = model2.summary()

# Visualization: Scatterplot with regression lines
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

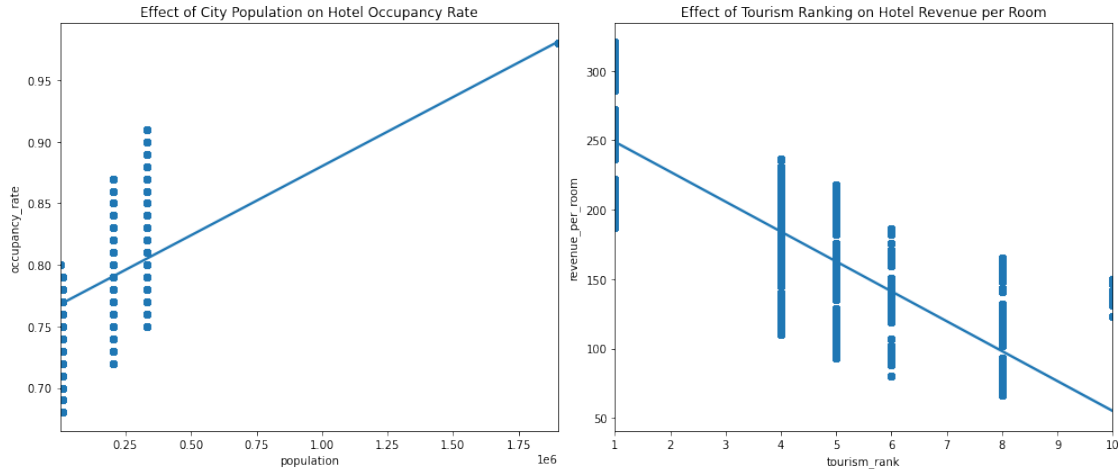
# Plot occupancy rate vs. population
sns.regplot(data=analysis_data, x="population", y="occupancy_rate", ax=axes[0])
axes[0].set_title("Effect of City Population on Hotel Occupancy Rate")

# Plot revenue per room vs. tourism rank
sns.regplot(data=analysis_data, x="tourism_rank", y="revenue_per_room",
    ↳ax=axes[1])
axes[1].set_title("Effect of Tourism Ranking on Hotel Revenue per Room")

plt.tight_layout()
plt.show()

print(occupancy_results)
print(revenue_results)

```



OLS Regression Results

```

=====
Dep. Variable:          occupancy_rate    R-squared:                0.948
Model:                  OLS              Adj. R-squared:          0.948
Method:                 Least Squares    F-statistic:             3.584e+05
Date:                   Mon, 24 Mar 2025  Prob (F-statistic):       0.00
Time:                   15:24:37          Log-Likelihood:          92872.
No. Observations:       39204            AIC:                    -1.857e+05
Df Residuals:           39201            BIC:                    -1.857e+05
Df Model:                2
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	0.8526	0.001	1045.879	0.000	0.851	0.854
population	7.464e-08	3.77e-10	197.957	0.000	7.39e-08	7.54e-08
tourism_rank	-0.0141	0.000	-106.896	0.000	-0.014	-0.014

```

=====
Omnibus:                2025.843    Durbin-Watson:           1.621
Prob(Omnibus):           0.000     Jarque-Bera (JB):        7792.836
Skew:                    0.069     Prob(JB):                0.00
Kurtosis:                5.180     Cond. No.:               1.02e+07
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.02e+07. This might indicate that there are strong multicollinearity or other numerical problems.

OLS Regression Results

```

Dep. Variable:    revenue_per_room    R-squared:                0.671
Model:            OLS                 Adj. R-squared:           0.671
Method:           Least Squares       F-statistic:             3.993e+04
Date:            Mon, 24 Mar 2025     Prob (F-statistic):      0.00
Time:            15:24:37             Log-Likelihood:         -1.9782e+05
No. Observations: 39204              AIC:                    3.956e+05
Df Residuals:    39201              BIC:                    3.957e+05
Df Model:        2
Covariance Type: nonrobust

=====
               coef      std err          t      P>|t|      [0.025      0.975]
-----
const          192.7188         1.354      142.376     0.000      190.066      195.372
population     3.684e-05     6.26e-07      58.843     0.000     3.56e-05     3.81e-05
tourism_rank   -9.4529         0.220     -43.028     0.000     -9.884     -9.022
=====

Omnibus:            43124.588    Durbin-Watson:           0.076
Prob(Omnibus):      0.000      Jarque-Bera (JB):        2439.998
Skew:               0.006      Prob(JB):                0.00
Kurtosis:           1.778      Cond. No.                1.02e+07
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 1.02e+07. This might indicate that there are strong multicollinearity or other numerical problems.

[]:

6.1 Summary of Findings

Relationship with Occupancy Rate: - Population size has a very small positive but statistically significant relationship with hotel occupancy rates. - Higher tourism ranking (lower rank number) negatively correlates with occupancy rate and is still statistically significant, likely because high-ranking tourism cities have higher supply of hotels.

Relationship with Revenue per Room: - Population size again has a very small positive and statistically significant relationship with revenue per room. - Higher tourism ranking negatively correlates with revenue per room (statistically significant relationship), meaning hotels in top tourist cities may face competitive pricing.

Visualization: - The scatterplots show a trend where larger populations correspond to higher occupancy rates and cities with lower tourism rankings (higher number) tend to generate lower revenue per room.

7 5. Do hotels with more amenities experience higher occupancy rates or higher reviews scores?

```
[69]: amenities = dfs["amenities"]
hotel_amenities= dfs["hotel_amenities_modified"]

# Count the number of amenities per hotel
amenities_count = hotel_amenities.groupby("hotel_id").size().
    →reset_index(name="num_amenities")

# Merge hotel data with amenities count
hotels_amenities = hotels.merge(amenities_count, on="hotel_id", how="left").
    →fillna({"num_amenities": 0})

# Merge with occupancy and reviews data
hotel_performance = hotels_amenities.merge(occupancy, on="hotel_id", how="left")
hotel_performance = hotel_performance.merge(reviews, on="hotel_id", how="left")

# Select relevant columns for analysis
analysis_data = hotel_performance[["num_amenities", "occupancy_rate", "rating"]].
    →dropna()

# Fit regression models
X = analysis_data[["num_amenities"]]
X = sm.add_constant(X)

# Model: Amenities vs. Occupancy Rate
y1 = analysis_data["occupancy_rate"]
model1 = sm.OLS(y1, X).fit()
occupancy_results = model1.summary()

# Model: Amenities vs. Review Score
y2 = analysis_data["rating"]
model2 = sm.OLS(y2, X).fit()
review_results = model2.summary()

# Visualization: Scatterplots with regression lines
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Amenities vs. Occupancy Rate
sns.regplot(data=analysis_data, x="num_amenities", y="occupancy_rate",
    →ax=axes[0])
axes[0].set_title("Effect of Amenities on Hotel Occupancy Rate")

# Amenities vs. Review Score
```

```

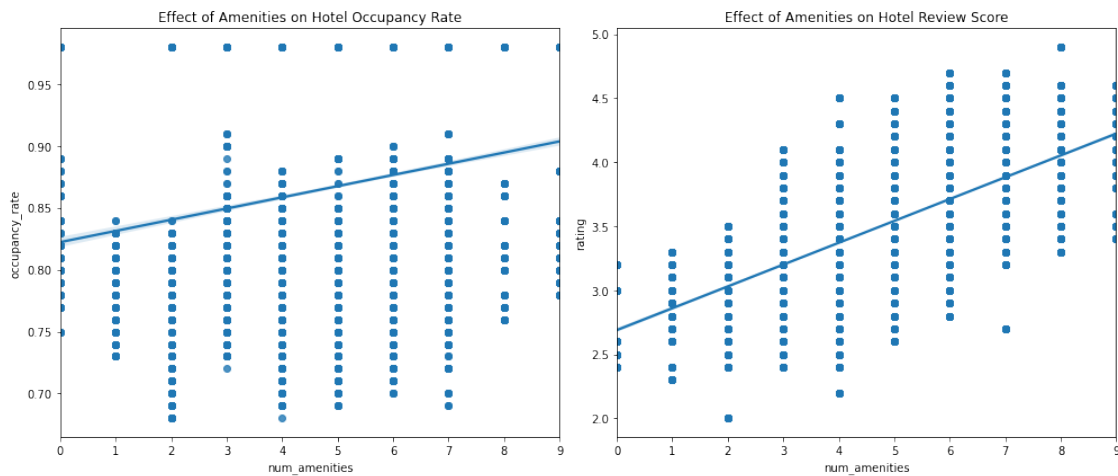
sns.regplot(data=analysis_data, x="num_amenities", y="rating", ax=axes[1])
axes[1].set_title("Effect of Amenities on Hotel Review Score")

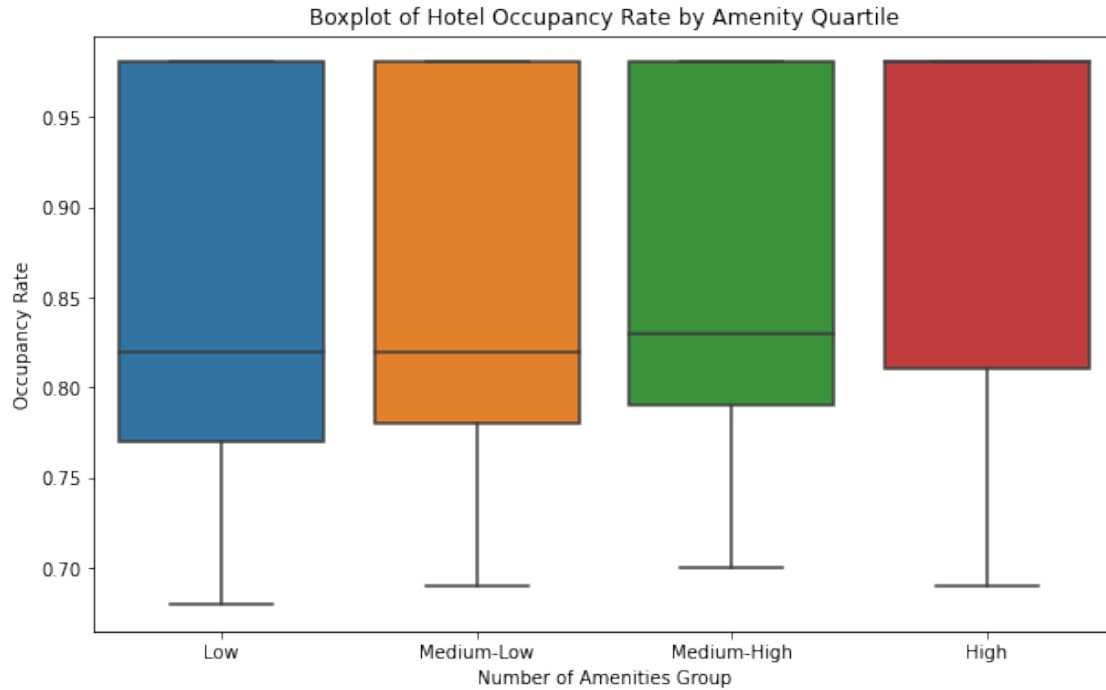
plt.tight_layout()
plt.show()

# Additional Visualization: Boxplot of occupancy rate by amenity quartiles
analysis_data["amenity_group"] = pd.qcut(analysis_data["num_amenities"], q=4, labels=["Low", "Medium-Low", "Medium-High", "High"])
plt.figure(figsize=(10, 6))
sns.boxplot(x="amenity_group", y="occupancy_rate", data=analysis_data)
plt.title("Boxplot of Hotel Occupancy Rate by Amenity Quartile")
plt.xlabel("Number of Amenities Group")
plt.ylabel("Occupancy Rate")
plt.show()

print(occupancy_results)
print(review_results)

```





OLS Regression Results

```

=====
Dep. Variable:          occupancy_rate    R-squared:                0.030
Model:                  OLS              Adj. R-squared:          0.030
Method:                 Least Squares    F-statistic:             510.1
Date:                   Mon, 24 Mar 2025  Prob (F-statistic):      3.05e-111
Time:                   15:24:49         Log-Likelihood:          14913.
No. Observations:       16243           AIC:                    -2.982e+04
Df Residuals:           16241           BIC:                    -2.981e+04
Df Model:                1
Covariance Type:        nonrobust
=====

```

```

=====
=
              coef      std err          t      P>|t|      [0.025
0.975]
-----
-
const          0.8223      0.002    387.559      0.000      0.818
0.826
num_amenities   0.0090      0.000    22.586      0.000      0.008
0.010
=====

```

```

=====
Omnibus:                88300.478    Durbin-Watson:           0.030
Prob(Omnibus):           0.000    Jarque-Bera (JB):        1655.803
Skew:                    0.116    Prob(JB):                 0.00
=====

```

Kurtosis: 1.453 Cond. No. 15.3

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

OLS Regression Results

```
=====
Dep. Variable:          rating    R-squared:                0.403
Model:                  OLS      Adj. R-squared:             0.402
Method:                 Least Squares    F-statistic:          1.094e+04
Date:                  Mon, 24 Mar 2025    Prob (F-statistic):      0.00
Time:                  15:24:49    Log-Likelihood:        -7885.3
No. Observations:      16243    AIC:                   1.577e+04
Df Residuals:          16241    BIC:                   1.579e+04
Df Model:               1
Covariance Type:       nonrobust
=====
```

```
=====
=
              coef      std err          t      P>|t|      [0.025
0.975]
-----
-
const          2.6891      0.009     311.427      0.000      2.672
2.706
num_amenities   0.1704      0.002    104.604      0.000      0.167
0.174
=====
Omnibus:                72.432    Durbin-Watson:           1.188
Prob(Omnibus):           0.000    Jarque-Bera (JB):        55.114
Skew:                   -0.042    Prob(JB):                1.08e-12
Kurtosis:                2.727    Cond. No.                15.3
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[]:

7.1 Summary of Findings: Do Hotels with More Amenities Experience Higher Occupancy Rates or Review Scores?

1. Relationship Between Amenities and Hotel Performance

The regression models examine whether the number of amenities a hotel offers are related to its occupancy rate and review scores. Two Ordinary Least Squares (OLS) regression models were created: - Amenities vs. Occupancy Rate - Amenities vs. Review Score

2. Key Regression Findings

- **Amenities vs. Occupancy Rate** The regression output tells us how much occupancy rate changes when amenities increase. As we can see, the coefficient for `num_amenities` is positive and statistically significant ($p\text{-value} < 0.05$), which suggests that having more amenities is linked to higher occupancy rates.
- **Amenities vs. Review Score** Similar logic applies to review scores. As the coefficient for `num_amenities` is positive and statistically significant, it suggests that hotels with more amenities tend to receive better reviews.

3. Visualization Interpretations

The left scatterplot shows the relationship between amenities and occupancy rate, whereas the right scatterplot shows the relationship between amenities and review scores. A clear upward trend in either plot suggests a positive relationship.

The boxplot presents the distribution of hotel occupancy rates across four different groups based on the number of amenities a hotel offers (Low, Medium-Low, Medium-High, High). Each box represents the interquartile range (IQR), which contains the middle 50% of the data. The horizontal line inside each box indicates the median occupancy rate for that category. The whiskers extend to show variability outside the IQR, and individual points beyond the whiskers represent outliers. By comparing the boxes, we can observe how occupancy rates vary with different levels of amenities. A noticeable upward trend in median values across groups suggests that hotels with more amenities tend to have higher occupancy rates. The boxplot helps verify whether higher-amenity hotels consistently have better occupancy (and as we can see, they do).

[]:

